1. Setup Minikube in your local machine.

Install minikube on windows:

==========

1) Install Docker on windows.

https://docs.docker.com/desktop/install/windows-install/

2) Install Oracle box on windows

https://adamtheautomator.com/install-virtualbox-on-windows-10/

3) Check system info in CMD

systeminfo

4) Install Minikube using powershell

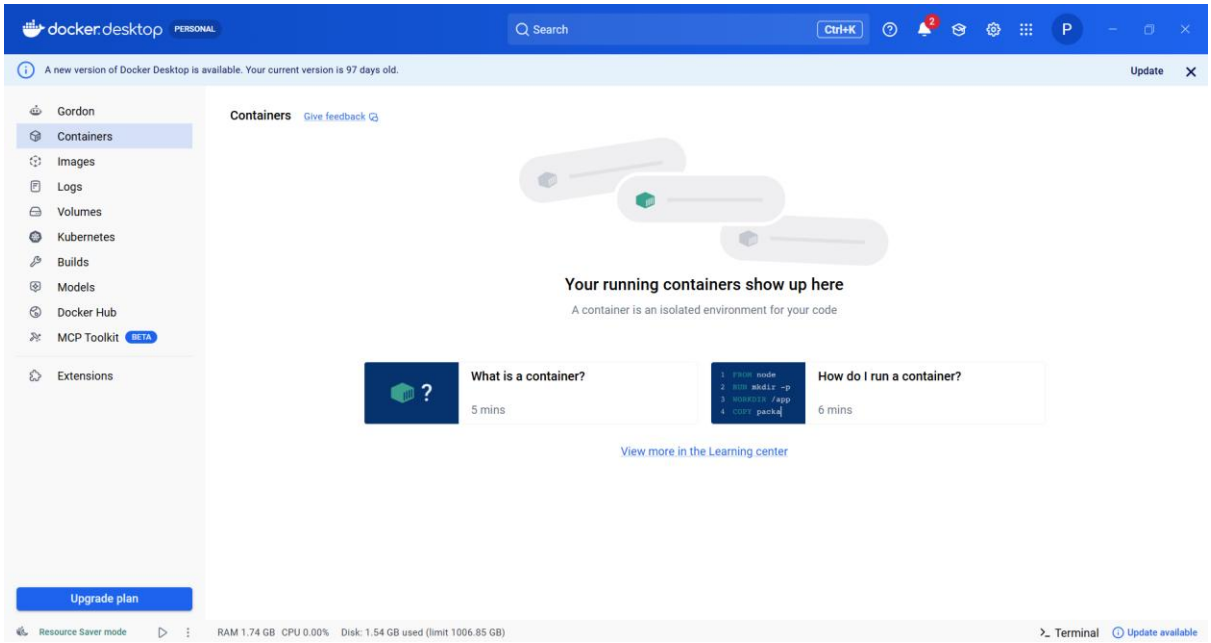
https://minikube.sigs.k8s.io/docs/start/

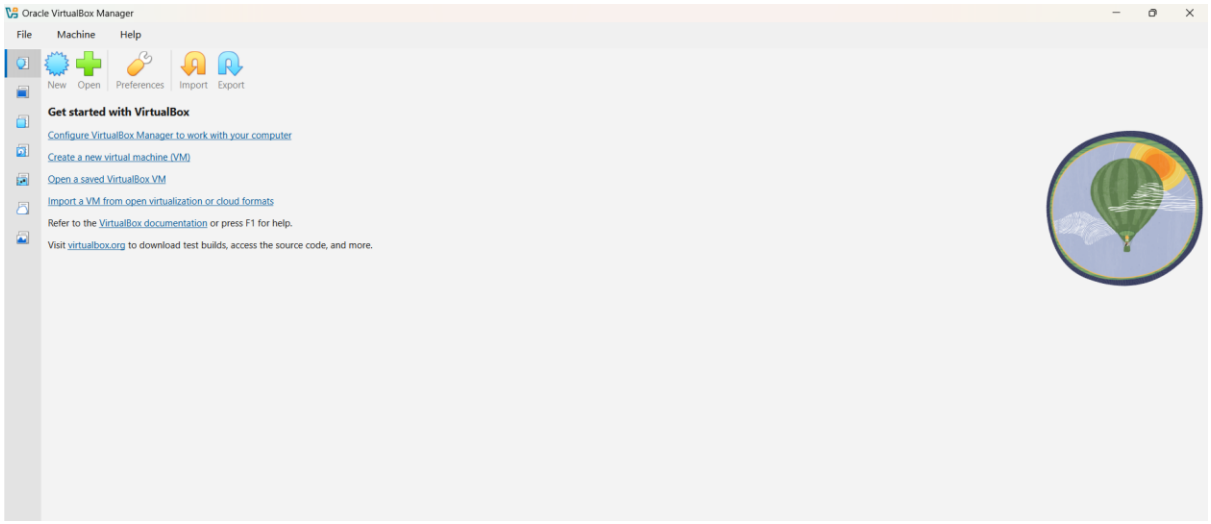
5) Set the minikube path to environmental varaible

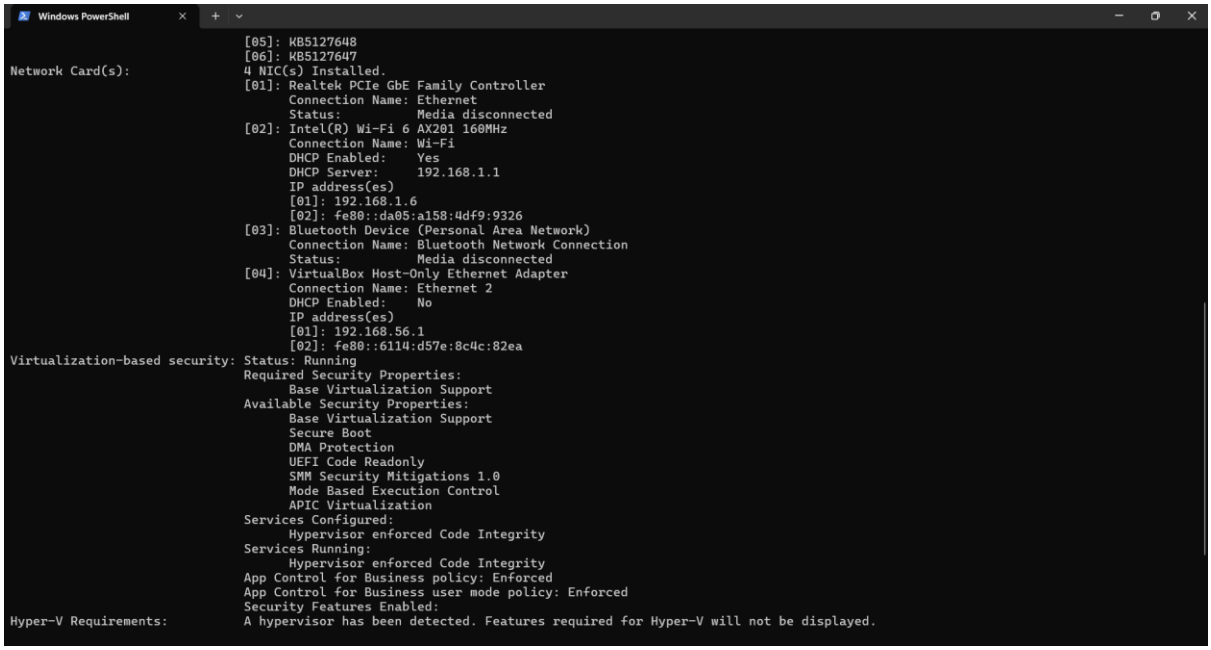
export PATH=$PATH:/c/minikube

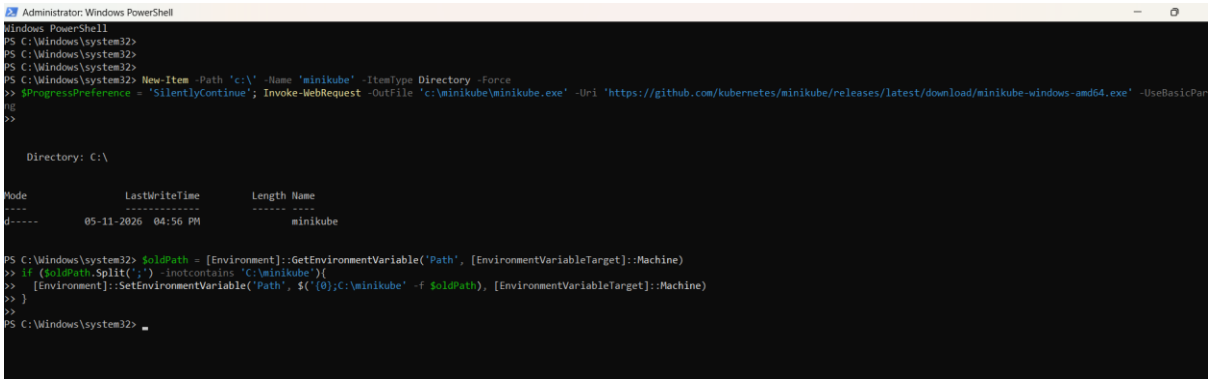
6) Restart powershell and start minikube

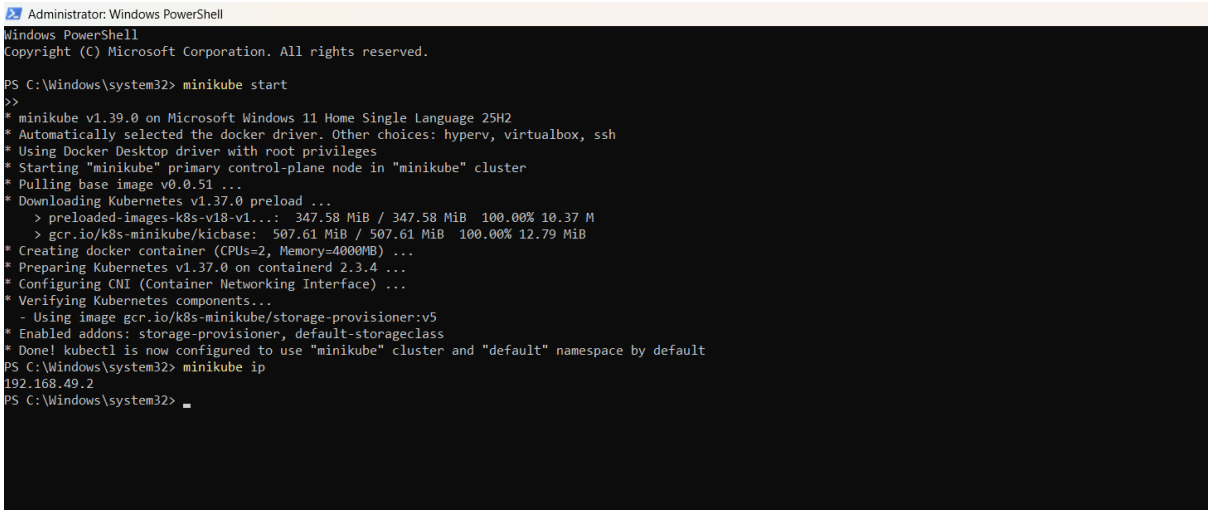
minikube start

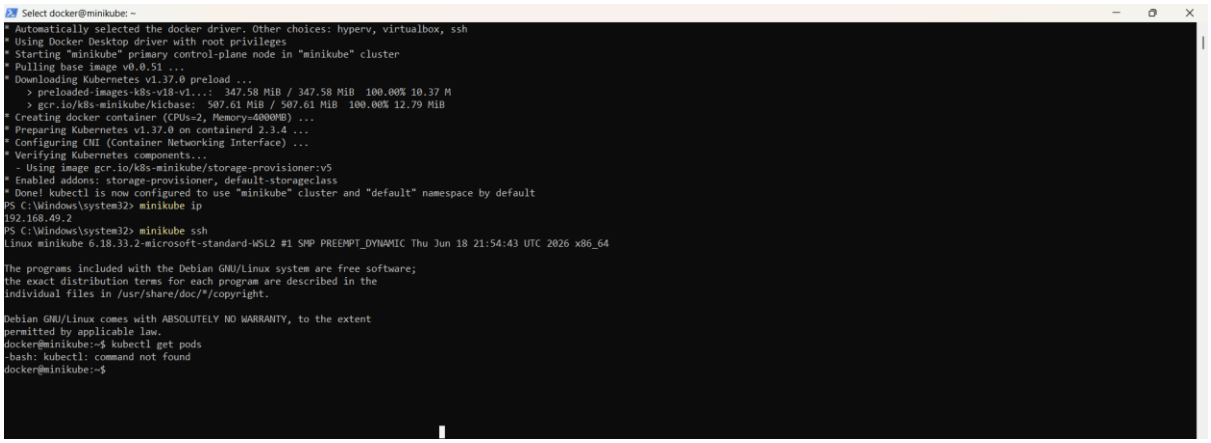


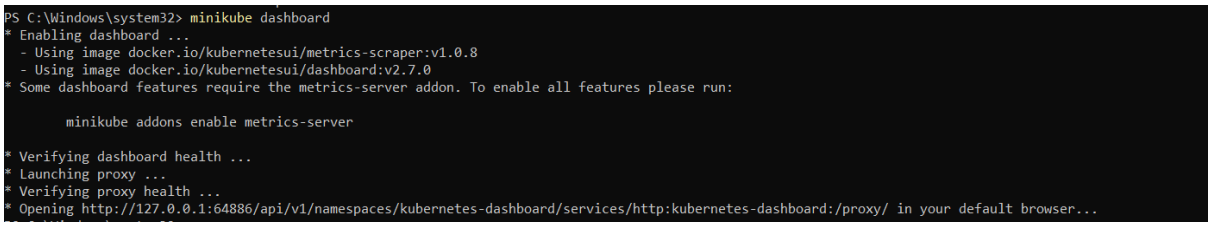

Oracle VirtualBox Manager
File
Machine
Help
New
Open
Preferences
Import
Export
Get started with VirtualBox
Configure VirtualBox Manager to work with your computer
Create a new virtual machine (VM)
Open a saved VirtualBox VM
Import a VM from open virtualization or cloud formats
Refer to the VirtualBox documentation or press F1 for help.
Visit virtualbox.org to download test builds, access the source code, and more.

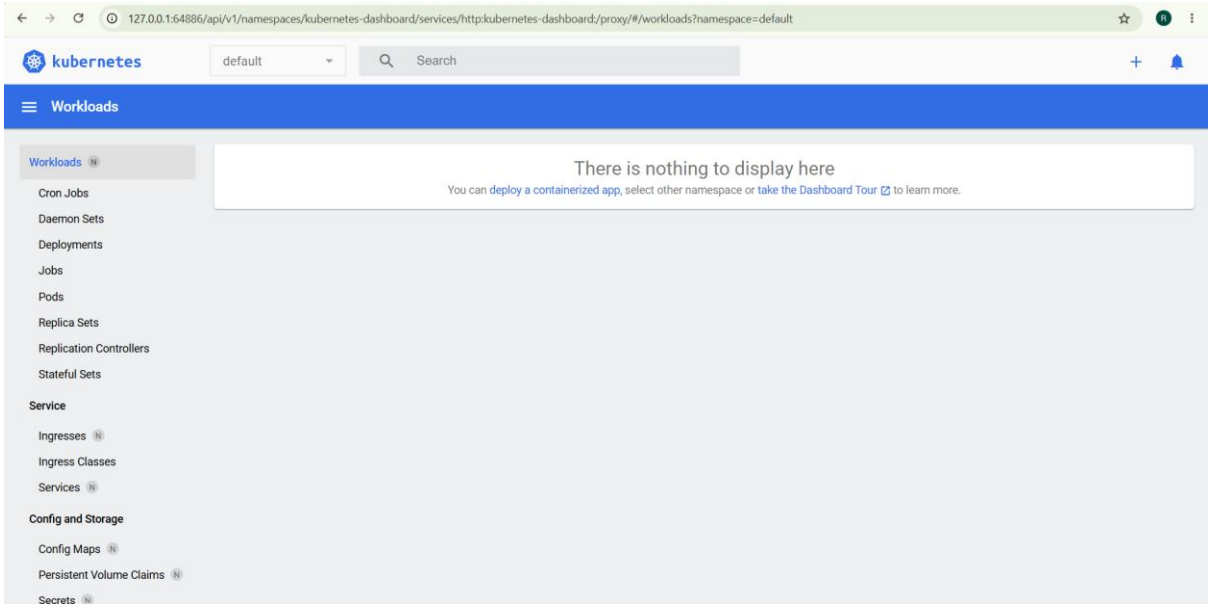

Windows PowerShell
[05]: KB5127648
[06]: KB5127647
Network Card(s): 4 NIC(s) Installed.
[01]: Realtek PCIe GbE Family Controller
Connection Name: Ethernet
Status: Media disconnected
[02]: Intel(R) Wi-Fi 6 AX201 160MHz
Connection Name: Wi-Fi
DHCP Enabled: Yes
DHCP Server: 192.168.1.1
IP address(es)
[01]: 192.168.1.6
[02]: fe80::da05:a158:4df9:9326
[03]: Bluetooth Device (Personal Area Network)
Connection Name: Bluetooth Network Connection
Status: Media disconnected
[04]: VirtualBox Host-Only Ethernet Adapter
Connection Name: Ethernet 2
DHCP Enabled: No
IP address(es)
[01]: 192.168.56.1
[02]: fe80::6114:d57e:8c4c:82ea
Virtualization-based security: Status: Running
Required Security Properties:
Base Virtualization Support
Available Security Properties:
Base Virtualization Support
Secure Boot
DMA Protection
UEFI Code Readonly
SMM Security Mitigations 1.0
Mode Based Execution Control
APIC Virtualization
Services Configured:
Hypervisor enforced Code Integrity
Services Running:
Hypervisor enforced Code Integrity
App Control for Business policy: Enforced
App Control for Business user mode policy: Enforced
Security Features Enabled:
Hyper-V Requirements: A hypervisor has been detected. Features required for Hyper-V will not be displayed.


Administrator: Windows PowerShell
Windows PowerShell
PS C:\Windows\system32>
PS C:\Windows\system32>
PS C:\Windows\system32>
PS C:\Windows\system32> New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force
>> $ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri 'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe' -UseBasicPar
sing
>>
Directory: C:\
Mode LastWriteTime Length Name
d----- 05-11-2026 04:56 PM minikube
PS C:\Windows\system32> $oldPath = [Environment]::GetEnvironmentVariable('Path', [EnvironmentVariableTarget]::Machine)
>> if ($oldPath.Split(';') -inotcontains 'C:\minikube'){
>> [Environment]::SetEnvironmentVariable('Path', $('{0};C:\minikube' -f $oldPath), [EnvironmentVariableTarget]::Machine)
>> }
>>
PS C:\Windows\system32>


Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Windows\system32> minikube start
>>
* minikube v1.39.0 on Microsoft Windows 11 Home Single Language 25H2
* Automatically selected the docker driver. Other choices: hyperv, virtualbox, ssh
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
> preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 10.37 M
> gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 12.79 MiB
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Windows\system32> minikube ip
192.168.49.2
PS C:\Windows\system32>


Select docker@minikube: ~
* Automatically selected the docker driver. Other choices: hyperv, virtualbox, ssh
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Downloading Kubernetes v1.37.0 preload ...
> preloaded-images-k8s-v18-v1...: 347.58 MiB / 347.58 MiB 100.00% 10.37 M
> gcr.io/k8s-minikube/kicbase: 507.61 MiB / 507.61 MiB 100.00% 12.79 MiB
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Configuring CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Windows\system32> minikube ip
192.168.49.2
PS C:\Windows\system32> minikube ssh
Linux minikube 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
docker@minikube:~$ kubectl get pods
-bash: kubectl: command not found
docker@minikube:~$


PS C:\Windows\system32> minikube dashboard
* Enabling dashboard ...
- Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
- Using image docker.io/kubernetesui/dashboard:v2.7.0
* Some dashboard features require the metrics-server addon. To enable all features please run:
minikube addons enable metrics-server
* Verifying dashboard health ...
* Launching proxy ...
* Verifying proxy health ...
* Opening http://127.0.0.1:64886/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...


127.0.0.1:64886/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/workloads?namespace=default
kubernetes
default
Search
Workloads
Workloads
Cron Jobs
Daemon Sets
Deployments
Jobs
Pods
Replica Sets
Replication Controllers
Stateful Sets
Service
Ingresses
Ingress Classes
Services
Config and Storage
Config Maps
Persistent Volume Claims
Secrets
There is nothing to display here
You can deploy a containerized app, select other namespace or take the Dashboard Tour to learn more.

2. **Setup k8s master and two worker nodes on ubuntu.**

Step 1: Launch EC2 instances

1. Logged in to the AWS Management Console.
2. Opened the EC2 dashboard.
3. Launched three Ubuntu EC2 instances.
4. Named the instances:
    - master
    - worker-01
    - worker-02
5. Selected the required instance type (t2.medium as instructed by the trainer).
6. Configured the security group to allow SSH and Kubernetes cluster communication.
7. Connected to all three instances using SSH.

Step 2: Disable Swap Memory

Executed the following commands on all three instances.

sudo swapoff -a

To disable swap permanently:

sudo sed -i '/ swap / s/^/#/' /etc/fstab

Verified using:

free -h

Step 3: Install Docker

Installed Docker on all three Ubuntu instances.

sudo apt update

sudo apt install -y docker.io

Started and enabled Docker:

sudo systemctl enable --now docker

Verified Docker:

sudo systemctl status docker

Step 4: Install Kubernetes Components

Configured the Kubernetes package repository on all three instances.

Installed the required packages:

sudo apt update

sudo apt install -y apt-transport-https ca-certificates curl gpg

Added the Kubernetes signing key and repository:

sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

Updated the repository and installed Kubernetes components:

sudo apt update

sudo apt install -y kubelet kubeadm kubectl

Held the package versions to prevent automatic upgrades:

sudo apt-mark hold kubelet kubeadm kubectl

Enabled kubelet:

sudo systemctl enable --now kubelet

Step 5: Configure Containerd

Configured containerd as the Kubernetes container runtime on all three instances.

Generated the default containerd configuration:

sudo mkdir -p /etc/containerd

sudo containerd config default | sudo tee /etc/containerd/config.toml

Changed the SystemdCgroup setting from false to true:

sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

Restarted and enabled containerd:

sudo systemctl restart containerd

sudo systemctl enable containerd

Verified:

sudo systemctl status containerd

Step 6: Initialize the Kubernetes Master

Connected to the Master instance.

Initialized the Kubernetes cluster with the Flannel network CIDR:

sudo kubeadm init --pod-network-cidr=10.244.0.0/16

After successful initialization, Kubernetes generated a kubeadm join command for adding the worker nodes.

Saved this command for later use.

Step 7: Configure kubectl on Master

Executed the following commands on the Master instance:

mkdir -p $HOME/.kube

sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config

sudo chown $(id -u):$(id -g) $HOME/.kube/config

Verified the Master node:

kubectl get nodes

Step 8: Install Flannel Network Plugin

Installed Flannel as the Container Network Interface (CNI) plugin.

kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml

Initially, the Flannel pod encountered an issue because the br_netfilter kernel module was missing.

Resolved it on the Master by executing:

sudo modprobe br_netfilter

Created the Kubernetes networking configuration:

sudo tee /etc/sysctl.d/k8s.conf <<EOF

net.bridge.bridge-nf-call-iptables = 1

net.bridge.bridge-nf-call-ip6tables = 1

net.ipv4.ip_forward = 1

EOF

Applied the settings:

sudo sysctl --system

Enabled the module permanently:

echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf

Deleted the failed Flannel pod so that Kubernetes could recreate it:

kubectl delete pod -n kube-flannel -l app=flannel

Verified the Flannel pod:

kubectl get pods -n kube-flannel

Confirmed that the Flannel pod was running successfully.

Step 9: Configure Networking on Worker Nodes

Performed the required networking configuration on the worker instances.

Loaded the br_netfilter module:

sudo modprobe br_netfilter

Created the networking configuration:

sudo tee /etc/sysctl.d/k8s.conf <<EOF

net.bridge.bridge-nf-call-iptables = 1

net.bridge.bridge-nf-call-ip6tables = 1

net.ipv4.ip_forward = 1

EOF

Applied the configuration:

sudo sysctl --system

Enabled the module permanently:

echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf

Step 10: Join Worker Nodes to the Master

Executed the kubeadm join command generated during Master initialization on Worker 1 and Worker 2.

Command used:

sudo kubeadm join 172.31.14.234:6443 --token <your-token> --discovery-token-ca-cert-hash sha256:<your-hash>

Both worker nodes successfully joined the cluster.

The terminal displayed:

This node has joined the cluster:

* Certificate signing request was sent to apiserver.

* The Kubelet was informed of the new secure connection details.

Step 11: Verify the Kubernetes Cluster

Returned to the Master instance and executed:

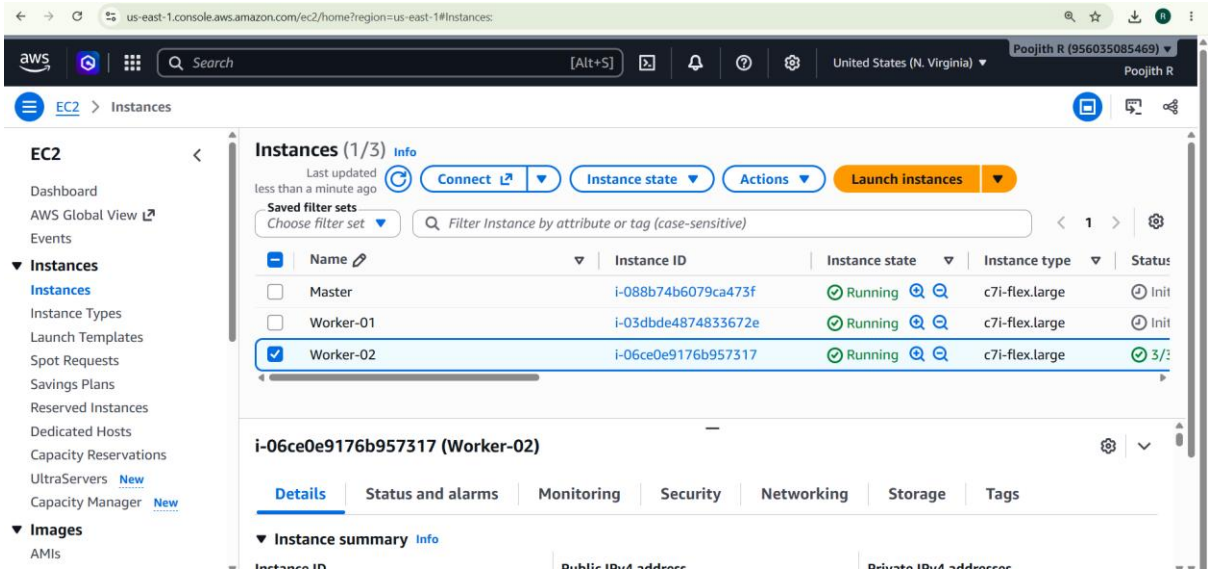
kubectl get nodes -o wide

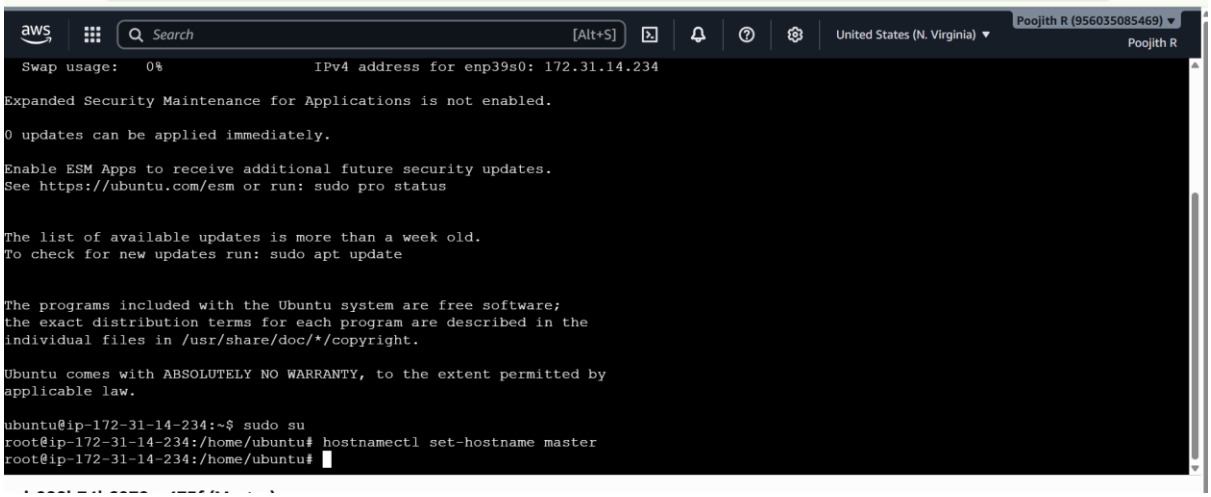
Verified all three nodes were in Ready status.

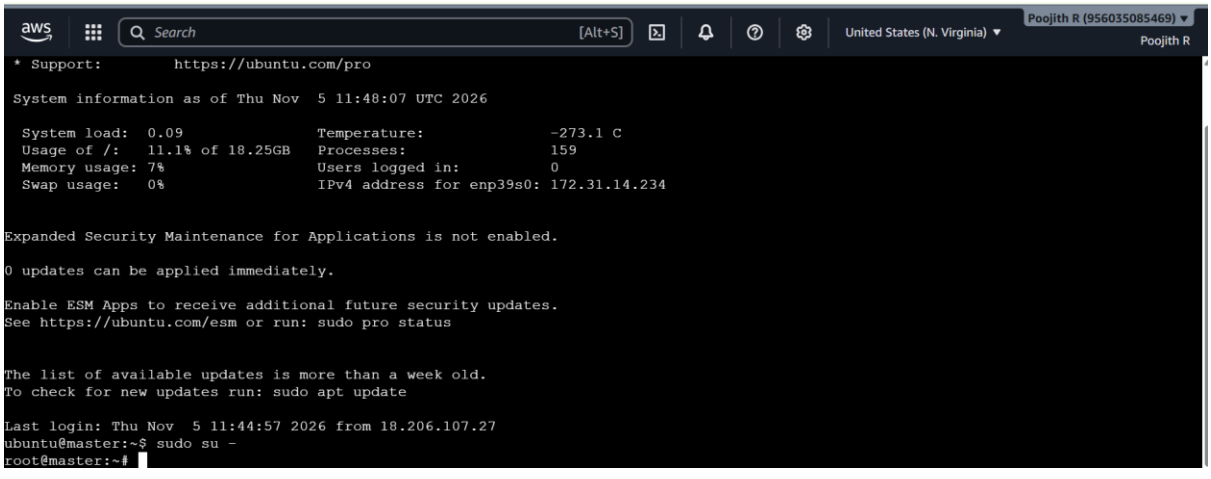
Also verified all system pods:

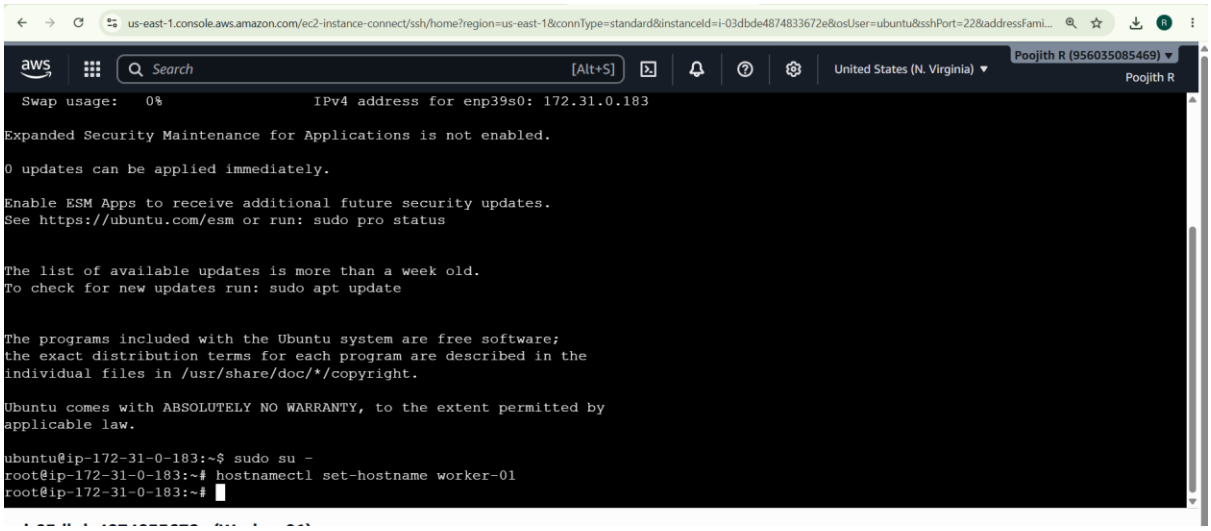
kubectl get pods -A

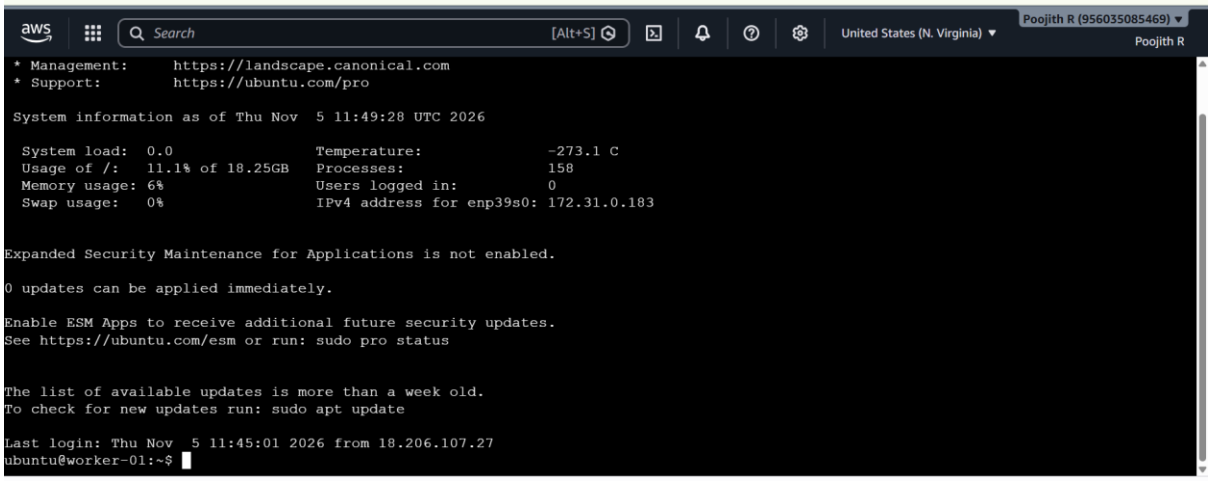
Confirmed that Flannel, CoreDNS, etcd, API Server, Controller Manager, Scheduler and kube-proxy were running.

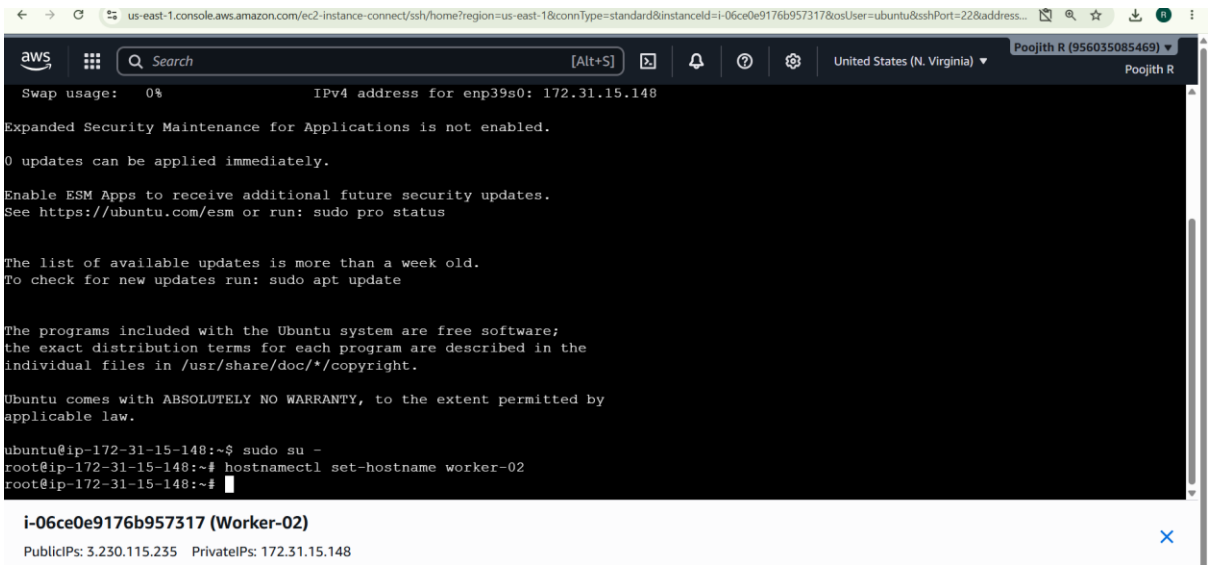

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
Instances (1/3) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Master
i-088b74b6079ca473f
Running
c7i-flex.large
Worker-01
i-03dbde4874833672e
Running
c7i-flex.large
Worker-02
i-06ce0e9176b957317
Running
c7i-flex.large
i-06ce0e9176b957317 (Worker-02)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info

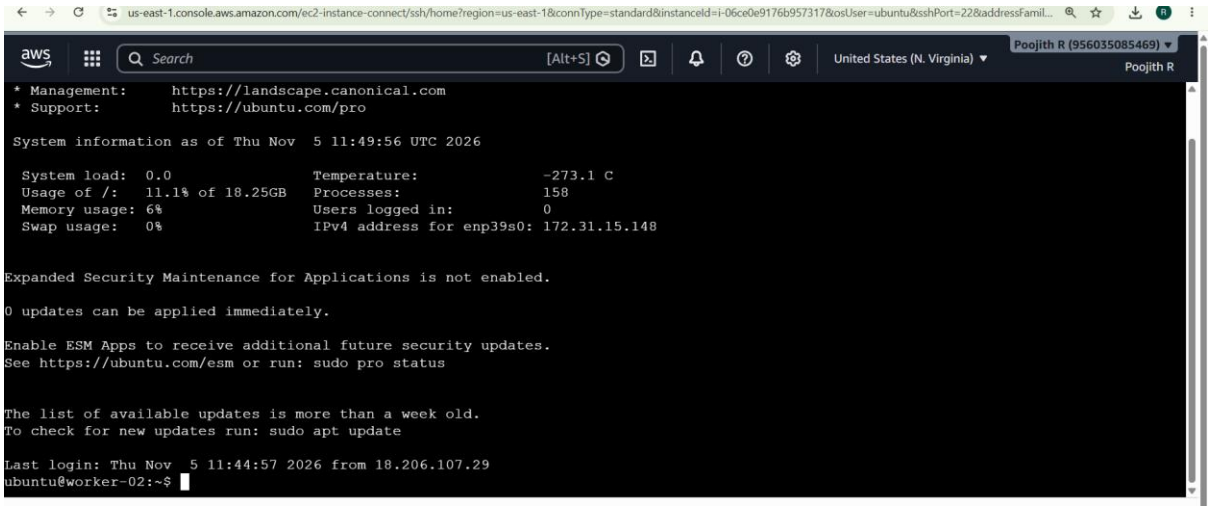

Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Swap usage: 0%
IPv4 address for enp39s0: 172.31.14.234
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ubuntu@ip-172-31-14-234:~$ sudo su
root@ip-172-31-14-234:/home/ubuntu# hostnamectl set-hostname master
root@ip-172-31-14-234:/home/ubuntu#


Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
* Support: https://ubuntu.com/pro
System information as of Thu Nov 5 11:48:07 UTC 2026
System load: 0.09
Temperature: -273.1 C
Usage of /: 11.1% of 18.25GB
Processes: 159
Memory usage: 7%
Users logged in: 0
Swap usage: 0%
IPv4 address for enp39s0: 172.31.14.234
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Thu Nov 5 11:44:57 2026 from 18.206.107.27
ubuntu@master:~$ sudo su -
root@master:~#

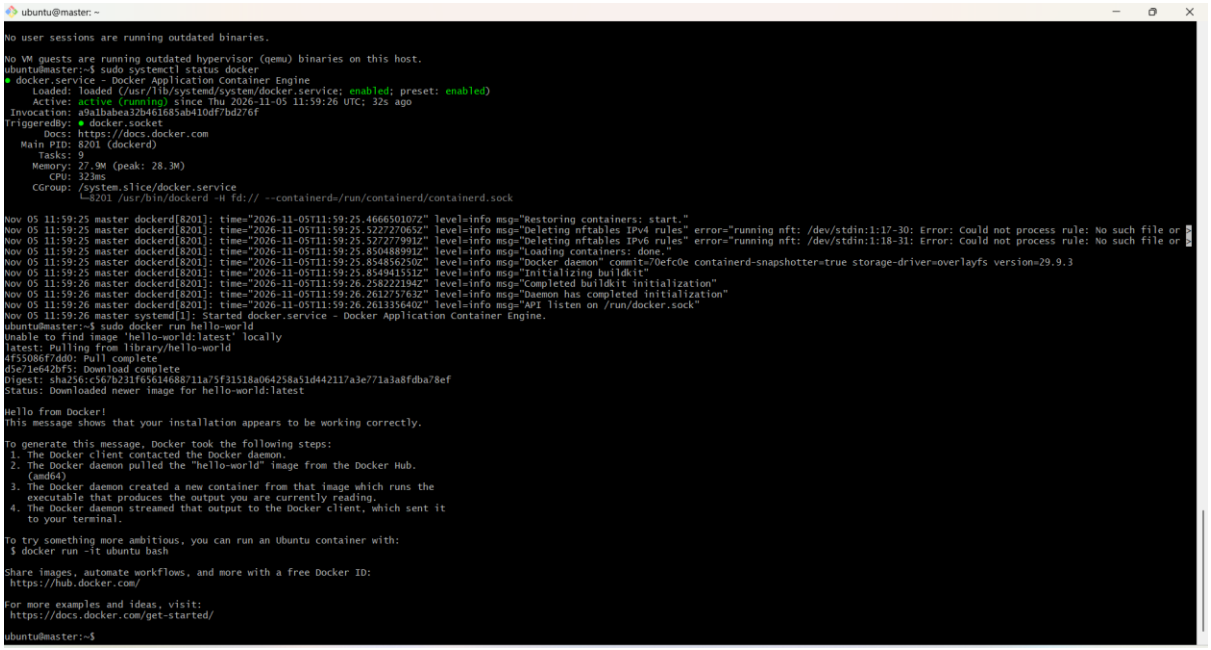

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-03dbde4874833672e&osUser=ubuntu&sshPort=22&addressFami...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Swap usage: 0% IPv4 address for enp39s0: 172.31.0.183
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ubuntu@ip-172-31-0-183:~$ sudo su -
root@ip-172-31-0-183:~# hostnamectl set-hostname worker-01
root@ip-172-31-0-183:~#

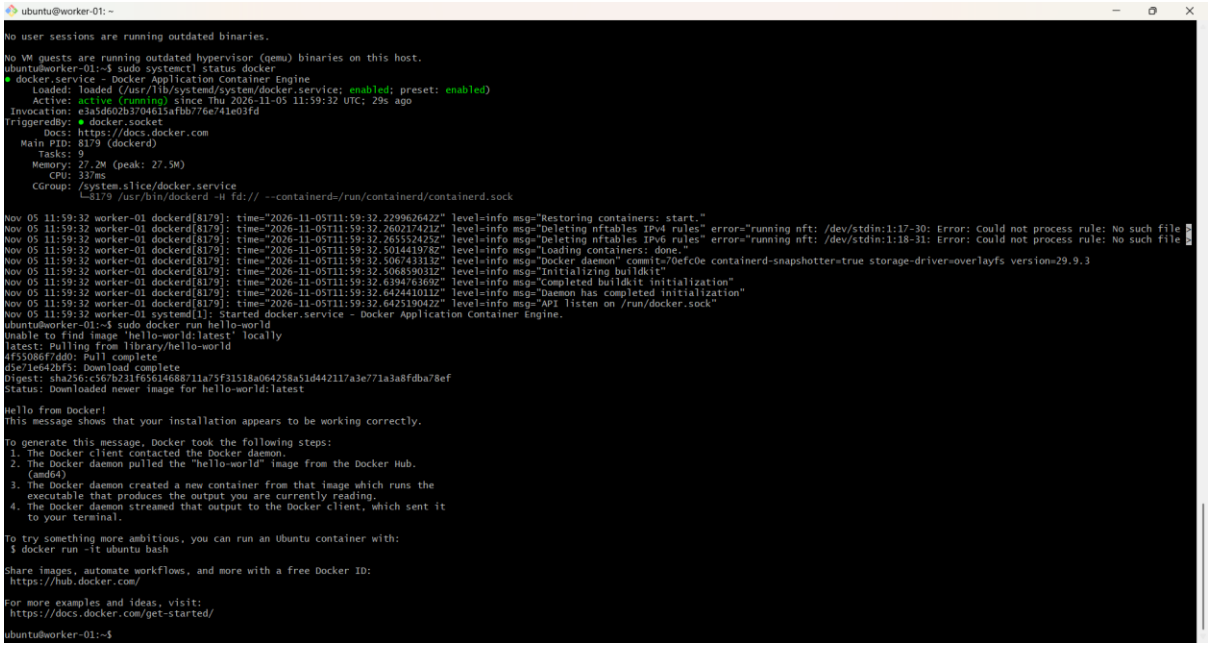

aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Thu Nov 5 11:49:28 UTC 2026
System load: 0.0 Temperature: -273.1 C
Usage of /: 11.1% of 18.25GB Processes: 158
Memory usage: 6% Users logged in: 0
Swap usage: 0% IPv4 address for enp39s0: 172.31.0.183
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Thu Nov 5 11:45:01 2026 from 18.206.107.27
ubuntu@worker-01:~$

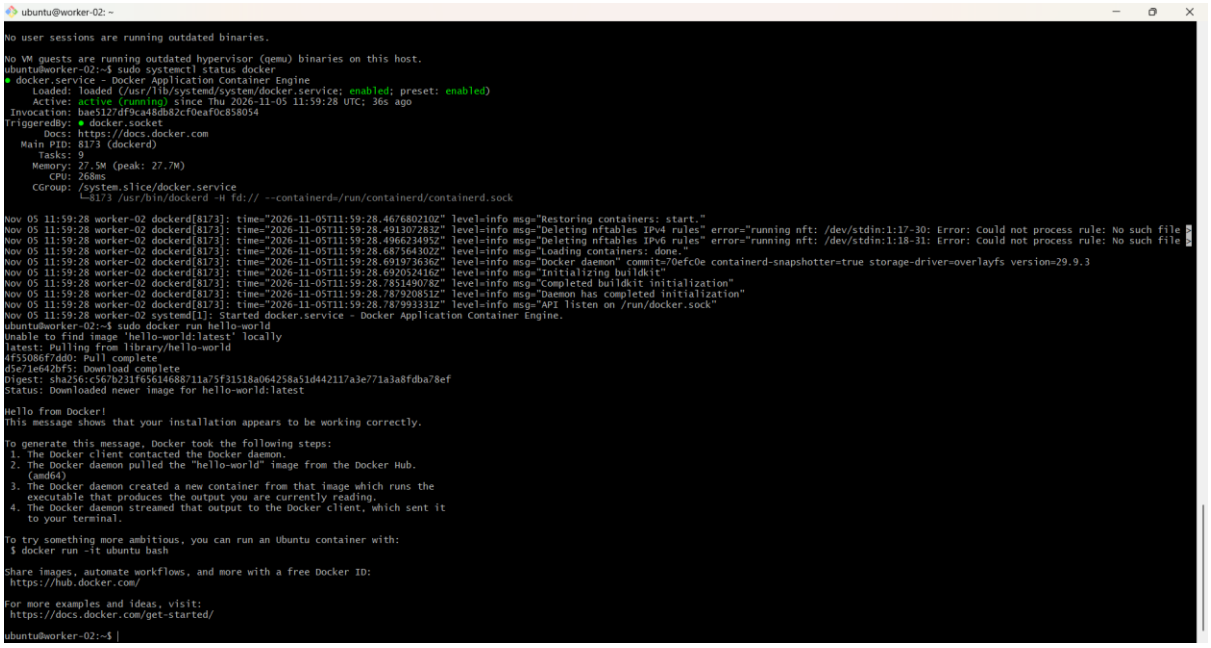

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-06ce0e9176b957317&osUser=ubuntu&sshPort=22&address...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Swap usage: 0% IPv4 address for enp39s0: 172.31.15.148
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ubuntu@ip-172-31-15-148:~$ sudo su -
root@ip-172-31-15-148:~# hostnamectl set-hostname worker-02
root@ip-172-31-15-148:~#
i-06ce0e9176b957317 (Worker-02)
PublicIPs: 3.230.115.235 PrivateIPs: 172.31.15.148



Docker Installation:




ubuntu@worker-01: ~
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@worker-01:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
Active: active (running) since Thu 2026-11-05 11:59:32 UTC; 29s ago
Invocation: e3a5d602b3704615afbb776e741e03fd
TriggeredBy: ● docker.socket
Docs: https://docs.docker.com
Main PID: 8179 (dockerd)
Tasks: 9
Memory: 27.2M (peak: 27.5M)
CPU: 337ms
CGroup: /system.slice/docker.service
└─8179 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.229962642Z" level=info msg="Restoring containers: start."
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.260217421Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:1:17-30: Error: Could not process rule: No such file
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.265552425Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:1:18-31: Error: Could not process rule: No such file
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.501441978Z" level=info msg="Loading containers: done."
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.506743313Z" level=info msg="Docker daemon" commit=70efc0e containerd-snapshotter=true storage-driver=overlayfs version=29.9.3
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.506859031Z" level=info msg="Initializing buildkit"
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.639476369Z" level=info msg="Completed buildkit initialization"
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.642441011Z" level=info msg="Daemon has completed initialization"
Nov 05 11:59:32 worker-01 dockerd[8179]: time="2026-11-05T11:59:32.642519042Z" level=info msg="API listen on /run/docker.sock"
Nov 05 11:59:32 worker-01 systemd[1]: Started docker.service - Docker Application Container Engine.
ubuntu@worker-01:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/
ubuntu@worker-01:~$

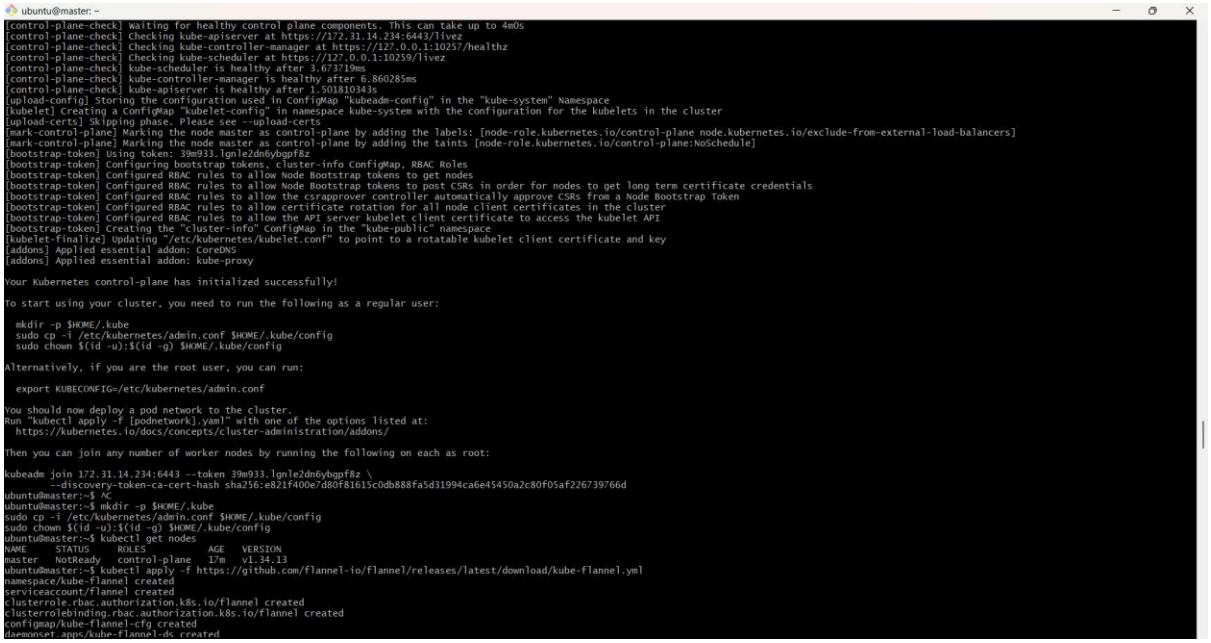

ubuntu@worker-02: ~
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@worker-02:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
Active: active (running) since Thu 2026-11-05 11:59:28 UTC; 36s ago
Invocation: bae5127df9ca48db82cf0eaf0c858054
TriggeredBy: ● docker.socket
Docs: https://docs.docker.com
Main PID: 8173 (dockerd)
Tasks: 9
Memory: 27.5M (peak: 27.7M)
CPU: 268ms
CGroup: /system.slice/docker.service
└─8173 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.467680210Z" level=info msg="Restoring containers: start."
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.491307283Z" level=info msg="Deleting nftables IPv4 rules" error="running nft: /dev/stdin:1:17-30: Error: Could not process rule: No such file
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.496623495Z" level=info msg="Deleting nftables IPv6 rules" error="running nft: /dev/stdin:1:18-31: Error: Could not process rule: No such file
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.687564302Z" level=info msg="Loading containers: done."
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.691973636Z" level=info msg="Docker daemon" commit=70efc0e containerd-snapshotter=true storage-driver=overlayfs version=29.9.3
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.692052416Z" level=info msg="Initializing buildkit"
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.785149078Z" level=info msg="Completed buildkit initialization"
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.787920851Z" level=info msg="Daemon has completed initialization"
Nov 05 11:59:28 worker-02 dockerd[8173]: time="2026-11-05T11:59:28.787993331Z" level=info msg="API listen on /run/docker.sock"
Nov 05 11:59:28 worker-02 systemd[1]: Started docker.service - Docker Application Container Engine.
ubuntu@worker-02:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/
ubuntu@worker-02:~$

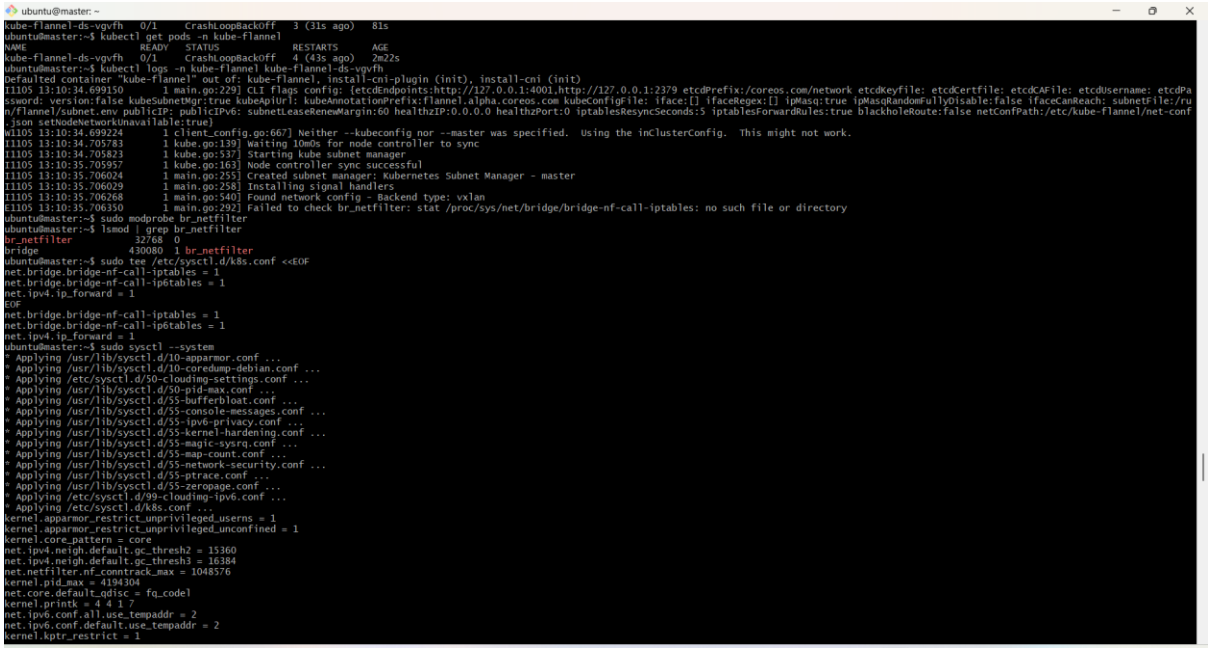
K8S setup:

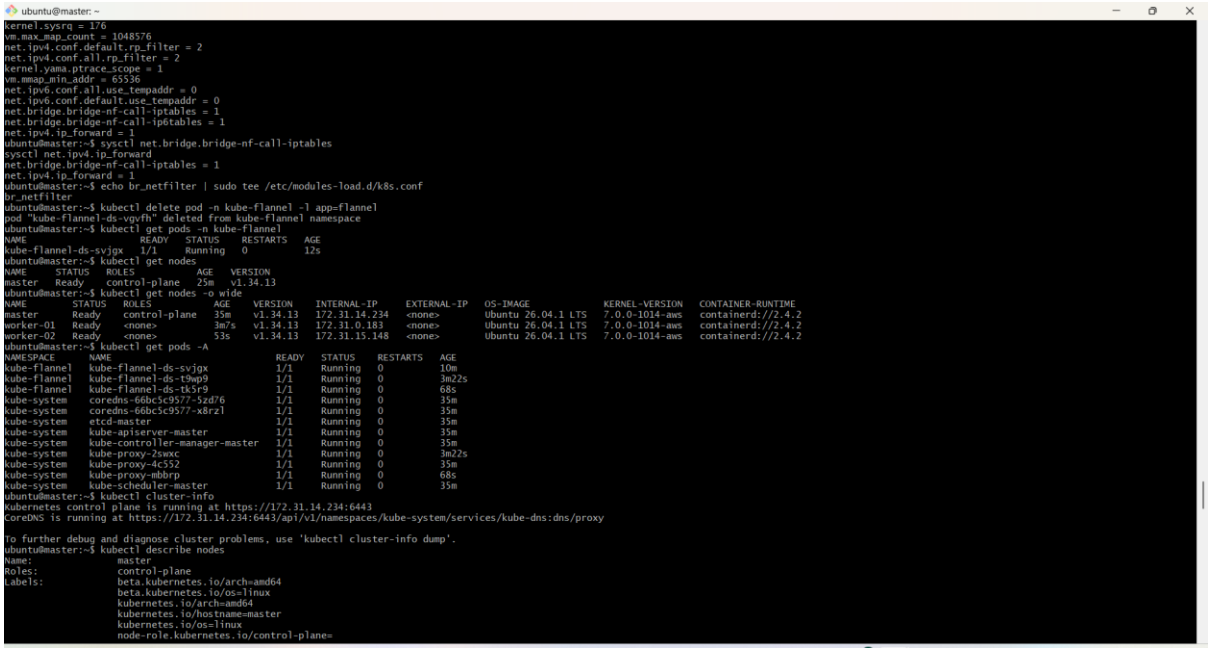

ubuntu@master:~$ sudo systemctl enable --now kubelet
ubuntu@master:~$ kubeadm version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.13", GitCommit:"5bafe0f04b114c29e7bda3bd2d08919a
8457b8d7", GitTreeState:"clean", BuildDate:"2026-10-27T12:05:17Z", GoVersion:"go1.26.10", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.13
Kustomize Version: v5.7.1
Kubernetes v1.34.13
ubuntu@master:~$

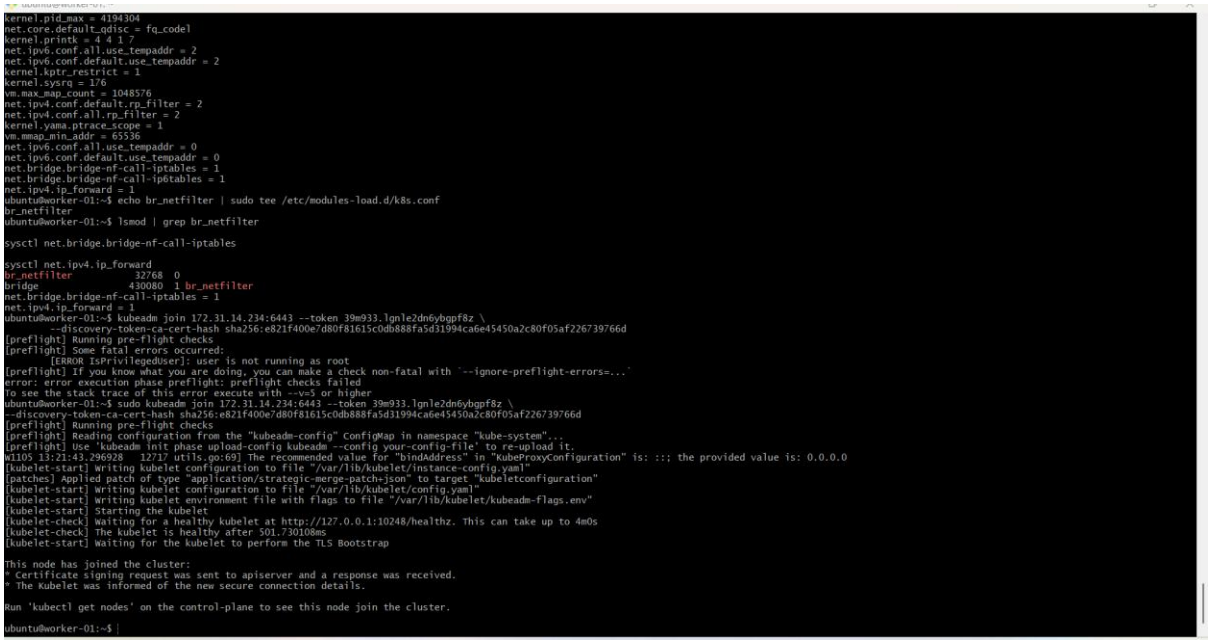

ubuntu@worker-02:~$ sudo systemctl enable --now kubelet
ubuntu@worker-02:~$ kubeadm version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.13", GitCommit:"5bafe0f04b114c29e7bda3bd2d08919a
8457b8d7", GitTreeState:"clean", BuildDate:"2026-10-27T12:05:17Z", GoVersion:"go1.26.10", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.13
Kustomize Version: v5.7.1
Kubernetes v1.34.13
ubuntu@worker-02:~$

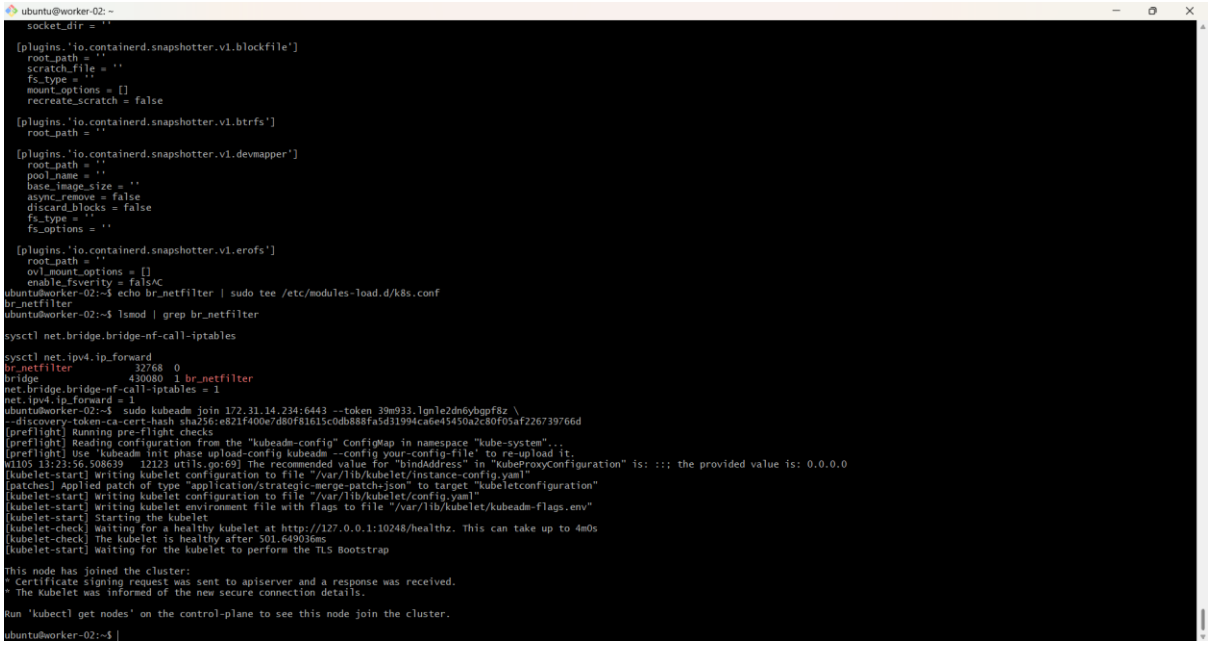

ubuntu@worker-02:~$ sudo systemctl enable --now kubelet
ubuntu@worker-02:~$ kubeadm version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"34", EmulationMajor:"", EmulationMinor:"", MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.34.13", GitCommit:"5bafe0f04b114c29e7bda3bd2d08919a
8457b8d7", GitTreeState:"clean", BuildDate:"2026-10-27T12:05:17Z", GoVersion:"go1.26.10", Compiler:"gc", Platform:"linux/amd64"}
Client Version: v1.34.13
Kustomize Version: v5.7.1
Kubernetes v1.34.13
ubuntu@worker-02:~$

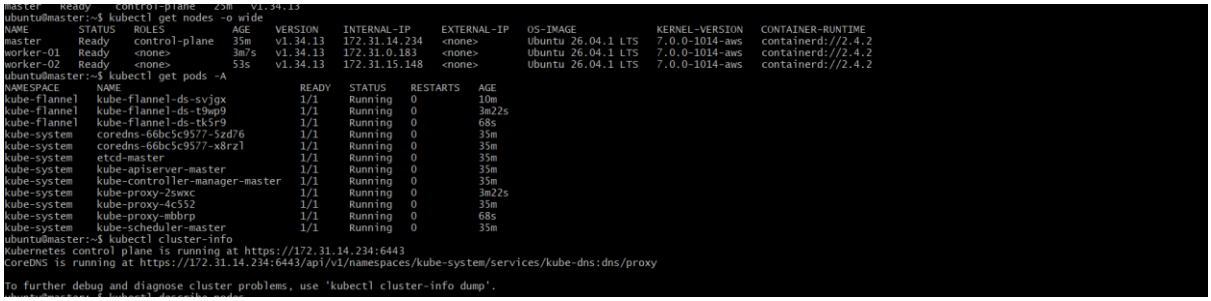

ubuntu@master: ~
[control-plane-check] Waiting for healthy control plane components. This can take up to 4m0s
[control-plane-check] Checking kube-apiserver at https://172.31.14.234:6443/livez
[control-plane-check] Checking kube-controller-manager at https://127.0.0.1:10257/healthz
[control-plane-check] Checking kube-scheduler at https://127.0.0.1:10259/livez
[control-plane-check] kube-scheduler is healthy after 3.673719ms
[control-plane-check] kube-controller-manager is healthy after 6.860285ms
[control-plane-check] kube-apiserver is healthy after 1.501810343s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the configuration for the kubelets in the cluster
[upload-certs] Skipping phase. Please see --upload-certs
[mark-control-plane] Marking the node master as control-plane by adding the labels: [node-role.kubernetes.io/control-plane node.kubernetes.io/exclude-from-external-load-balancers]
[mark-control-plane] Marking the node master as control-plane by adding the taints [node-role.kubernetes.io/control-plane:NoSchedule]
[bootstrap-token] Using token: 39m933.lgnle2dn6ybgpf8z
[bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC Roles
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get nodes
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in order for nodes to get long term certificate credentials
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller automatically approve CSRs from a Node Bootstrap Token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node client certificates in the cluster
[bootstrap-token] Configured RBAC rules to allow the API server kubelet client certificate to access the kubelet API
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" namespace
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy
Your Kubernetes control-plane has initialized successfully!
To start using your cluster, you need to run the following as a regular user:
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
Alternatively, if you are the root user, you can run:
export KUBECONFIG=/etc/kubernetes/admin.conf
You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/
Then you can join any number of worker nodes by running the following on each as root:
kubeadm join 172.31.14.234:6443 --token 39m933.lgnle2dn6ybgpf8z \
--discovery-token-ca-cert-hash sha256:e821f400e7d80f81615c0db888fa5d31994ca6e45450a2c80f05af226739766d
ubuntu@master:~$ AC
ubuntu@master:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
ubuntu@master:~$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
master NotReady control-plane 17m v1.34.13
ubuntu@master:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created


ubuntu@master: ~
kube-flannel-ds-vgvfh 0/1 CrashLoopBackOff 3 (31s ago) 81s
ubuntu@master:~$ kubectl get pods -n kube-flannel
NAME READY STATUS RESTARTS AGE
kube-flannel-ds-vgvfh 0/1 CrashLoopBackOff 4 (43s ago) 2m22s
ubuntu@master:~$ kubectl logs -n kube-flannel kube-flannel-ds-vgvfh
Defaulted container "kube-flannel" out of: kube-flannel, install-cni-plugin (init), install-cni (init)
I1105 13:10:34.699150 1 main.go:229] CLI flags config: {etcdEndpoints:http://127.0.0.1:4001,http://127.0.0.1:2379 etcdPrefix:/coreos.com/network etcdKeyfile: etcdCertfile: etcdCAFile: etcdUsername: etcdPassword: version:false kubeSubnetMgr:true kubeApiUrl: kubeAnnotationPrefix:flannel.alpha.coreos.com kubeConfigFile: iface:[] ifaceRegex:[] ipMasq:true ipMasqRandomFullyDisable:false ifaceCanReach: subnetFile:/run/flannel/subnet.env publicIP: publicIPv6: subnetLeaseRenewMargin:60 healthzIP:0.0.0.0 healthzPort:0 iptablesResyncSeconds:5 iptablesForwardRules:true blackholeRoute:false netConfPath:/etc/kube-flannel/net-conf.json setNodeNetworkUnavailable:true}
W1105 13:10:34.699224 1 client_config.go:667] Neither --kubeconfig nor --master was specified. Using the inClusterConfig. This might not work.
I1105 13:10:34.705783 1 kube.go:139] Waiting 10m0s for node controller to sync
I1105 13:10:34.705823 1 kube.go:537] Starting kube subnet manager
I1105 13:10:35.705957 1 kube.go:163] Node controller sync successful
I1105 13:10:35.706024 1 main.go:255] Created subnet manager: Kubernetes Subnet Manager - master
I1105 13:10:35.706029 1 main.go:258] Installing signal handlers
I1105 13:10:35.706268 1 main.go:540] Found network config - Backend type: vxlan
E1105 13:10:35.706350 1 main.go:292] Failed to check br_netfilter: stat /proc/sys/net/bridge/bridge-nf-call-iptables: no such file or directory
ubuntu@master:~$ sudo modprobe br_netfilter
ubuntu@master:~$ lsmod | grep br_netfilter
br_netfilter 32768 0
bridge 430080 1 br_netfilter
ubuntu@master:~$ sudo tee /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
ubuntu@master:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /usr/lib/sysctl.d/10-coredump-debian.conf ...
* Applying /etc/sysctl.d/50-cloudimg-settings.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /usr/lib/sysctl.d/55-bufferbloat.conf ...
* Applying /usr/lib/sysctl.d/55-console-messages.conf ...
* Applying /usr/lib/sysctl.d/55-ipv6-privacy.conf ...
* Applying /usr/lib/sysctl.d/55-kernel-hardening.conf ...
* Applying /usr/lib/sysctl.d/55-magic-sysrq.conf ...
* Applying /usr/lib/sysctl.d/55-map-count.conf ...
* Applying /usr/lib/sysctl.d/55-network-security.conf ...
* Applying /usr/lib/sysctl.d/55-ptrace.conf ...
* Applying /usr/lib/sysctl.d/55-zeropage.conf ...
* Applying /etc/sysctl.d/99-cloudimg-ipv6.conf ...
* Applying /etc/sysctl.d/k8s.conf ...
kernel.apparmor_restrict_unprivileged_userns = 1
kernel.apparmor_restrict_unprivileged_unconfined = 1
kernel.core_pattern = core
net.ipv4.neigh.default.gc_thresh2 = 15360
net.ipv4.neigh.default.gc_thresh3 = 16384
net.netfilter.nf_conntrack_max = 1048576
kernel.pid_max = 4194304
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1


ubuntu@master: ~
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.default.use_tempaddr = 0
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
ubuntu@master:~$ sysctl net.bridge.bridge-nf-call-iptables
sysctl net.ipv4.ip_forward
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
ubuntu@master:~$ echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf
br_netfilter
ubuntu@master:~$ kubectl delete pod -n kube-flannel -l app=flannel
pod "kube-flannel-ds-vgvfh" deleted from kube-flannel namespace
ubuntu@master:~$ kubectl get pods -n kube-flannel
NAME READY STATUS RESTARTS AGE
kube-flannel-ds-svjgx 1/1 Running 0 12s
ubuntu@master:~$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
master Ready control-plane 25m v1.34.13
ubuntu@master:~$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME
master Ready control-plane 35m v1.34.13 172.31.14.234 <none> Ubuntu 26.04.1 LTS 7.0.0-1014-aws containerd://2.4.2
worker-01 Ready <none> 3m7s v1.34.13 172.31.0.183 <none> Ubuntu 26.04.1 LTS 7.0.0-1014-aws containerd://2.4.2
worker-02 Ready <none> 53s v1.34.13 172.31.15.148 <none> Ubuntu 26.04.1 LTS 7.0.0-1014-aws containerd://2.4.2
ubuntu@master:~$ kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-flannel kube-flannel-ds-svjgx 1/1 Running 0 10m
kube-flannel kube-flannel-ds-t9wp9 1/1 Running 0 3m22s
kube-flannel kube-flannel-ds-tk5r9 1/1 Running 0 68s
kube-system coredns-66bc5c9577-5zd76 1/1 Running 0 35m
kube-system coredns-66bc5c9577-x8rzl 1/1 Running 0 35m
kube-system etcd-master 1/1 Running 0 35m
kube-system kube-apiserver-master 1/1 Running 0 35m
kube-system kube-controller-manager-master 1/1 Running 0 35m
kube-system kube-proxy-2swxc 1/1 Running 0 3m22s
kube-system kube-proxy-4c552 1/1 Running 0 35m
kube-system kube-proxy-mbbrp 1/1 Running 0 68s
kube-system kube-scheduler-master 1/1 Running 0 35m
ubuntu@master:~$ kubectl cluster-info
Kubernetes control plane is running at https://172.31.14.234:6443
CoreDNS is running at https://172.31.14.234:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
ubuntu@master:~$ kubectl describe nodes
Name: master
Roles: control-plane
Labels: beta.kubernetes.io/arch=amd64
beta.kubernetes.io/os=linux
kubernetes.io/arch=amd64
kubernetes.io/hostname=master
kubernetes.io/os=linux
node-role.kubernetes.io/control-plane=


kernel.pid_max = 4194304
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.default.use_tempaddr = 0
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
ubuntu@worker-01:~$ echo br_netfilter | sudo tee /etc/modules-load.d/k8s.conf
br_netfilter
ubuntu@worker-01:~$ lsmod | grep br_netfilter
sysctl net.bridge.bridge-nf-call-iptables
sysctl net.ipv4.ip_forward
br_netfilter 32768 0
bridge 430080 1 br_netfilter
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
ubuntu@worker-01:~$ kubeadm join 172.31.14.234:6443 --token 39m933.lgnle2dn6ybgpf8z \
--discovery-token-ca-cert-hash sha256:e821f400e7d80f81615c0db888fa5d31994ca6e45450a2c80f05af226739766d
[preflight] Running pre-flight checks
[preflight] Some fatal errors occurred:
[ERROR IsPrivilegedUser]: user is not running as root
[preflight] If you know what you are doing, you can make a check non-fatal with `--ignore-preflight-errors=...`
error: error execution phase preflight: preflight checks failed
To see the stack trace of this error execute with --v=5 or higher
ubuntu@worker-01:~$ sudo kubeadm join 172.31.14.234:6443 --token 39m933.lgnle2dn6ybgpf8z \
--discovery-token-ca-cert-hash sha256:e821f400e7d80f81615c0db888fa5d31994ca6e45450a2c80f05af226739766d
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
W1105 13:21:43.296928 12717 utils.go:69] The recommended value for "bindAddress" in "KubeProxyConfiguration" is: ::; the provided value is: 0.0.0.0
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.730108ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap
This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.
Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
ubuntu@worker-01:~$





3. Run one nginx pod.

Step 1: Create an Nginx Pod

Connected to the Kubernetes Master and executed:

kubectl run nginx --image=nginx

This command created a Pod named nginx using the official Nginx Docker image.

Step 2: Verify the Pod

Executed:

kubectl get pods

Verified that the Nginx pod was running.

Expected output:

NAME READY STATUS RESTARTS AGE

nginx 1/1 Running 0 30s

Step 3: Check Pod Details

Executed:

kubectl get pods -o wide

This displayed:

- Pod name
- Ready status
- Pod IP address
- Worker node where the pod was scheduled

Step 4: Describe the Nginx Pod

Executed:

kubectl describe pod nginx

Verified the following details:

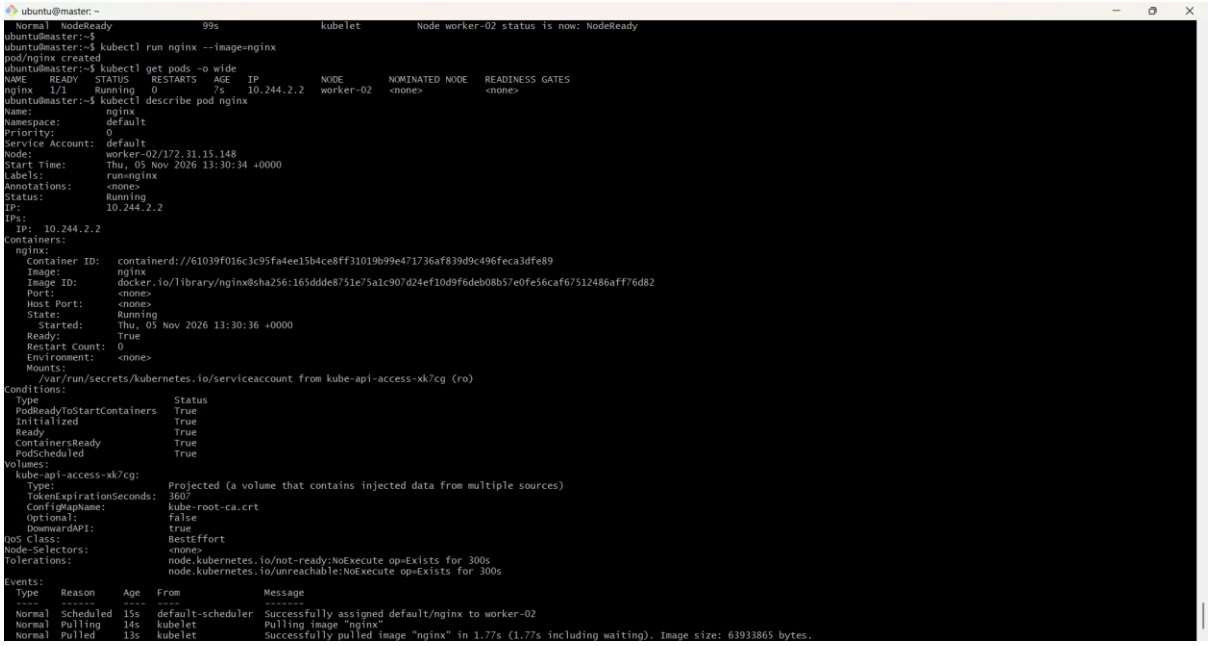
- Pod name: nginx
- Image: nginx
- Status: Running
- Ready: True
- Node: Worker node on which the pod was scheduled

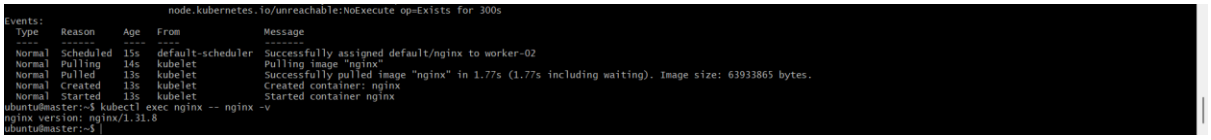
Step 5: Verify Nginx Version

Executed:

kubectl exec nginx -- nginx -v

This verified that Nginx was installed and running inside the container.





4. Mugup Master and slave components on k8s.

Objective

To understand the architecture, components, functions, and working mechanism of Kubernetes Master (Control Plane) and Worker Nodes.

1. Introduction to Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, management, and monitoring of containerized applications.

It provides features such as:

- Automated application deployment
- Scaling of applications
- Load balancing
- Self-healing
- Rolling updates and rollbacks
- Resource management

2. Kubernetes Architecture

A Kubernetes cluster consists of two major parts:

1. Control Plane (Master Node): Responsible for managing and controlling the Kubernetes cluster.

2. Worker Nodes: Responsible for running application workloads in the form of Pods.

The Control Plane manages the cluster, while Worker Nodes execute the applications.

3. Master Node (Control Plane) Components

The Control Plane is responsible for maintaining the desired state of the cluster and managing the Worker Nodes.

3.1 API Server (kube-apiserver)

The API Server is the central communication component of Kubernetes. It acts as the entry point for all API requests and interactions with the cluster.

Functions:

- Accepts requests from kubectl and other components.
- Validates and processes API requests.
- Communicates with etcd to store and retrieve cluster data.
- Enables communication between Kubernetes components.

Example: When the user executes kubectl get nodes, the request is processed by the API Server, which returns the node information.

3.2 etcd

etcd is a distributed key-value database used by Kubernetes to store cluster data and configuration.

Functions:

- Stores cluster state and configuration.
- Maintains information about Nodes, Pods, Secrets, ConfigMaps, and other Kubernetes objects.
- Acts as the source of truth for the cluster state.
- Supports reliable storage of Kubernetes data.

Example: When a new Pod is created, its desired configuration is stored in etcd.

3.3 Scheduler (kube-scheduler)

The Scheduler is responsible for assigning newly created Pods to suitable Worker Nodes.

Functions:

- Identifies unscheduled Pods.
- Checks available CPU, memory, and other resource requirements.
- Considers scheduling constraints and policies.
- Selects the appropriate Worker Node for a Pod.

Example: When an Nginx Pod is created, the Scheduler selects a suitable Worker Node based on resource availability and scheduling requirements.

3.4 Controller Manager (kube-controller-manager)

The Controller Manager runs various controllers that continuously monitor the cluster and maintain its desired state.

Functions:

- Monitors the actual state of the cluster.
- Compares the actual state with the desired state.
- Takes corrective actions when differences are detected.
- Ensures that resources are maintained as configured.

Example: If a Deployment is configured with three replicas and one Pod fails, the controller works to create a replacement Pod to maintain the desired number of replicas.

3.5 Cloud Controller Manager (cloud-controller-manager)

The Cloud Controller Manager integrates Kubernetes with cloud platforms such as AWS, Azure, and Google Cloud.

Functions:

- Integrates Kubernetes with cloud infrastructure.
- Manages cloud-specific node information.
- Supports cloud load balancer provisioning.
- Manages cloud routes and related infrastructure.

Example: In AWS, a Kubernetes Service of type LoadBalancer can trigger the provisioning of an external AWS load balancer when the appropriate cloud integration is configured.

4. Worker Node Components

Worker Nodes are responsible for executing and maintaining application workloads assigned by the Control Plane.

4.1 Kubelet

Kubelet is an agent that runs on every Kubernetes node.

Functions:

- Communicates with the API Server.
- Receives Pod specifications assigned to its node.
- Ensures that containers inside the assigned Pods are running.
- Monitors Pod and node health.
- Reports node and Pod status to the Control Plane.

Example: When a Pod is scheduled on Worker Node 1, the Kubelet ensures that the required containers are started and maintained.

4.2 Kube-proxy

Kube-proxy is a network component that runs on Kubernetes nodes and implements Service networking.

Functions:

- Maintains network rules on nodes.
- Enables communication between Services and backend Pods.
- Routes Service traffic to appropriate Pod endpoints.
- Supports network connectivity within the cluster.

Example: When a client accesses a Kubernetes Service, kube-proxy's networking rules help forward the traffic to an appropriate backend Pod.

4.3 Container Runtime

The Container Runtime is responsible for running and managing containers on Kubernetes nodes.

Functions:

- Pulls container images from registries.
- Creates and starts containers.
- Stops and manages container lifecycle.
- Provides container execution capabilities to Kubernetes through the Container Runtime Interface (CRI).

Examples:

- containerd
- CRI-O

In our practical implementation: We configured containerd as the container runtime on all three Ubuntu EC2 instances.

5. Difference Between Master and Worker Nodes

Master Node (Control Plane)	Worker Node
Manages the Kubernetes cluster	Runs application workloads
Controls cluster operations	Executes assigned Pods
Runs API Server	Runs Kubelet
Runs etcd	Runs Container Runtime
Runs Scheduler	Runs kube-proxy
Runs Controller Manager	Reports node and Pod status

6. Working Mechanism of Kubernetes

When a user deploys an application in Kubernetes, the following process takes place:

1. The user submits a request using kubectl.
2. The request reaches the API Server.
3. The API Server validates the request and stores the desired state in etcd.
4. The Scheduler identifies a suitable Worker Node for the Pod.
5. The scheduling decision is recorded through the API Server.
6. The Kubelet on the selected Worker Node receives the Pod specification.
7. Kubelet communicates with the Container Runtime to pull the image and start the container.

8. The Kubelet monitors the container and reports its status to the API Server.
9. The application Pod reaches the Running state.

7. Important Kubernetes Commands

Command	Description
kubectl get nodes	Displays cluster nodes
kubectl get pods -A	Displays Pods across all namespaces
kubectl get pods -n kube-system	Displays system Pods
kubectl get pods -n kube-flannel	Displays Flannel networking Pods
kubectl describe node master	Displays detailed Master node information
kubectl describe node worker-01	Displays detailed Worker node information
sudo systemctl status kubelet	Checks Kubelet service status
sudo systemctl status containerd	Checks container runtime status

8. Conclusion

Kubernetes follows a distributed architecture in which the Control Plane manages the cluster and Worker Nodes execute application workloads. The API Server, etcd, Scheduler, and Controller Manager are essential Control Plane components, while Kubelet, kube-proxy, and the Container Runtime are essential Worker Node components.

Understanding these components helps in managing, deploying, scaling, and troubleshooting applications in a Kubernetes cluster.